



河南大學
HENAN UNIVERSITY



Web编程基础

(2020-2021-2)

任课教师: 丁爽



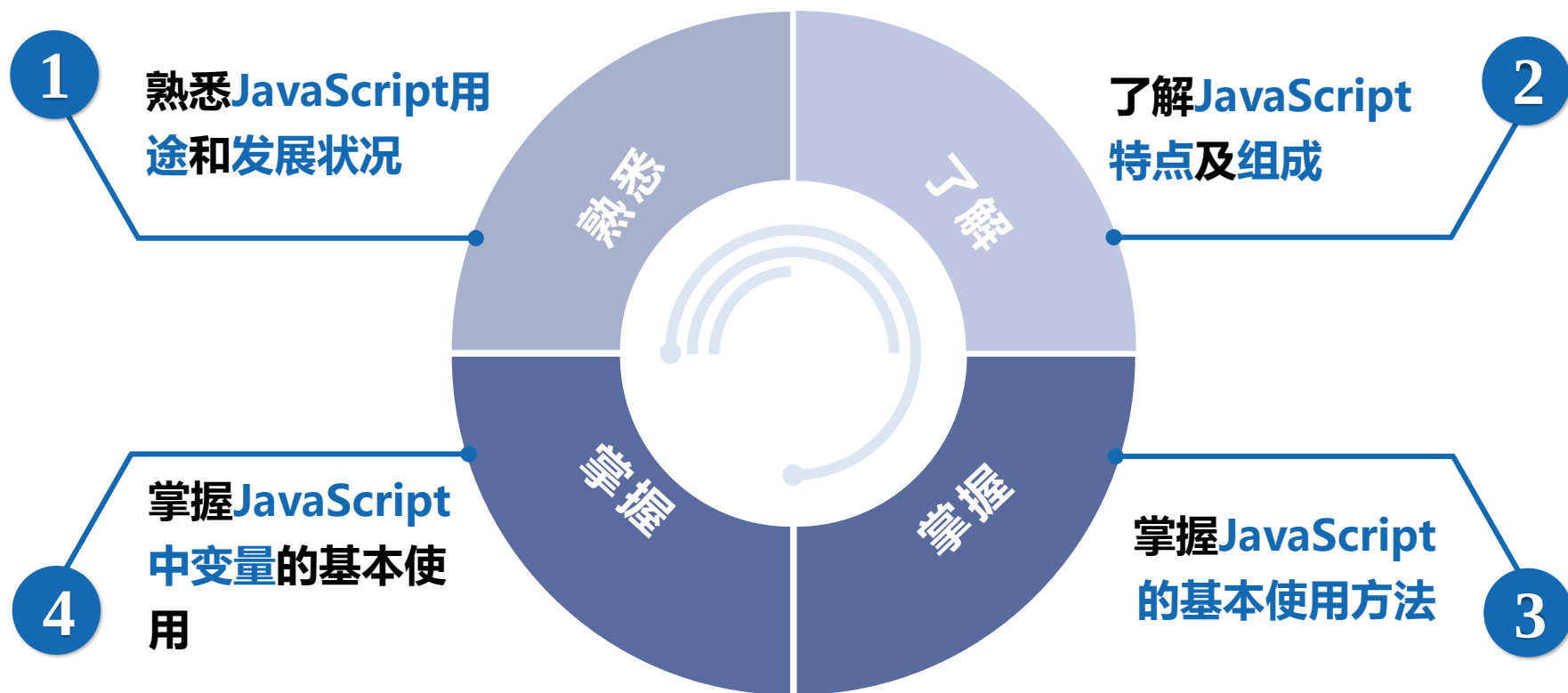
内容回顾

- 浮动：
 - 浮动 **float**: **left**、**right**、**none**
 - 清除浮动: **clear**: **left**、**right**、**both**
- 定位
 - **position**: **static**、**relative**、**absolute**、**fixed**
 - 边偏移: **top**、**bottom**、**left**、**right**
- 布局类型: 单列、双列、三列
- 命名规则: 驼峰式、帕斯卡



第1章 初识JavaScript

学习目标



目录



1.1

什么是JavaScript

[点击查看本节相关知识点](#)

1.2

常用开发工具

[点击查看本节相关知识点](#)

1.3

JavaScript入门

[点击查看本节相关知识点](#)

1.4

JavaScript变量

[点击查看本节相关知识点](#)



1.1 什么是JavaScript

1

JavaScript概述

JavaScript: 是Web开发领域中的一种功能强大的编程语言，主要用于开发交互式的Web页面。在计算机、手机等设备上浏览的网页，其大多数的交互逻辑几乎都是由JavaScript实现的。



1.1 什么是JavaScript

1

JavaScript概述

网页组成：对于制作一个网页而言，HTML、CSS和JavaScript分别代表了结构、样式和行为，三者之间的区别如下。

- HTML：决定网页的结构和内容，相当于人的身体。
- CSS：决定网页呈现给用户的模样，相当于给人穿衣服、化妆。
- JavaScript：实现业务逻辑和页面控制，相当于人的各种动作。



1.1 什么是JavaScript

1

JavaScript概述

JavaScript内嵌于HTML网页中，通过浏览器内置的JavaScript引擎进行解释执行，把一个原本只用来显示的页面转变成支持用户交互的页面程序。

浏览器是访问互联网中各种网站所必备的工具，JavaScript主要就是运行在浏览器中的。



1.1 什么是JavaScript

1

JavaScript概述

常见浏览器及特点：

- ❑ Internet Explorer: Windows操作系统的内置浏览器，用户数量较多
- ❑ Microsoft Edge: Windows10操作系统提供的浏览器，速度较快、功能较多
- ❑ Google Chrome: 目前市场占有率较高的浏览器，具有简洁、快速的特点
- ❑ Mozilla Firefox: 一款优秀的浏览器，市场占有率低于Google Chrome
- ❑ Safari: 主要应用在苹果iOS、macOS操作系统中的浏览器



1.1 什么是JavaScript

1

JavaScript概述

浏览器内核的分类：

- ❑ **渲染引擎（也称为排版引擎）**：负责解析HTML与CSS，如Chrome浏览器的Blink。
- ❑ **JavaScript引擎**：是JavaScript语言的解释器，用于读取网页中的JavaScript代码，对其处理后执行，如Chrome浏览器的V8引擎

JavaScript代码的执行特点：**逐行执行**



1.1 什么是JavaScript

2

JavaScript的诞生和发展

诞生：在1995年时，**Netscape**（网景）公司（现在的Mozilla）的布兰登·爱奇（Brendan Eich）在网景导航者浏览器上首次设计出了JavaScript。Netscape最初将这个脚本语言命名为LiveScript，后来Netscape公司与Sun公司合作之后将其改名为JavaScript。



1.1 什么是JavaScript

2

JavaScript的诞生和发展

在设计之初，JavaScript是一种可以嵌入到网页中的编程语言，用来控制浏览器的行为。例如，直接在浏览器中进行表单验证，用户只有填写格式正确的内容后才能够提交表单，这样避免了用户因表单填写错误导致的反复提交，节省了时间和网络资源，示例图如下。



1.1 什么是JavaScript

2 JavaScript的诞生和发展

填写注册信息

用户名称：

长度4~12，英文大小写字母

密 码：

长度6~20，大小写字母、数字或下划线

确认密码：

请再次输入密码进行确认

手机号码：

13、14、15、17、18开头的11位手机号

电子邮箱：

用户名@域名（域名后缀至少2个字符）

注册

表单验证



1.1 什么是JavaScript

2

JavaScript的诞生和发展

JavaScript的用途: 可以嵌入到网页中的编程语言，用来控制浏览器的行为。

案例应用如下：

- ❑ 利用前后端分离模式进行开发，基于React技术栈开发的移动Web项目
- ❑ 基于Angular构建的响应式移动Web项目
- ❑ 使用WePY框架并结合ES 6语法开发小程序电商项目
- ❑ 基于Vue.js全套技术栈开发出来的网页端后台管理系统



1.1 什么是JavaScript

2 JavaScript的诞生和发展



基于React开发的租房项目



基于Angular开发的移动Web预定酒店项目



1.1 什么是JavaScript

2 JavaScript的诞生和发展

权限管理

商品管理

商品列表

分类参数

商品分类

订单管理

数据统计

添加分类

#	分类名称	是否有效	排序	操作
1	大家电	✓	一级	编辑 删除
	电视	✓	二级	编辑 删除
	空调	✓	二级	编辑 删除
	洗衣机	✓	二级	编辑 删除
	洗衣机	✓	三级	编辑 删除
	滚筒洗衣机	✓	三级	编辑 删除
	波轮洗衣机	✓	三级	编辑 删除
	大容量洗衣机	✓	三级	编辑 删除
	迷你洗衣机	✓	三级	编辑 删除
	双缸洗衣机	✓	三级	编辑 删除
	干衣机	✓	三级	编辑 删除

Vue.js开发的后台管理系统项目



小程序电商项目



1.1 什么是JavaScript

3

JavaScript的特点

JavaScript特点如下:

- ❑ JavaScript是一种脚本语言，特点是简单、易学、易用，语法规则比较松散，能够快速完成程序的编写工作
- ❑ JavaScript可以跨平台，它不依赖操作系统，仅需要浏览器的支持
- ❑ JavaScript支持面向对象，可以使JavaScript开发变得快捷和高效，降低开发成本

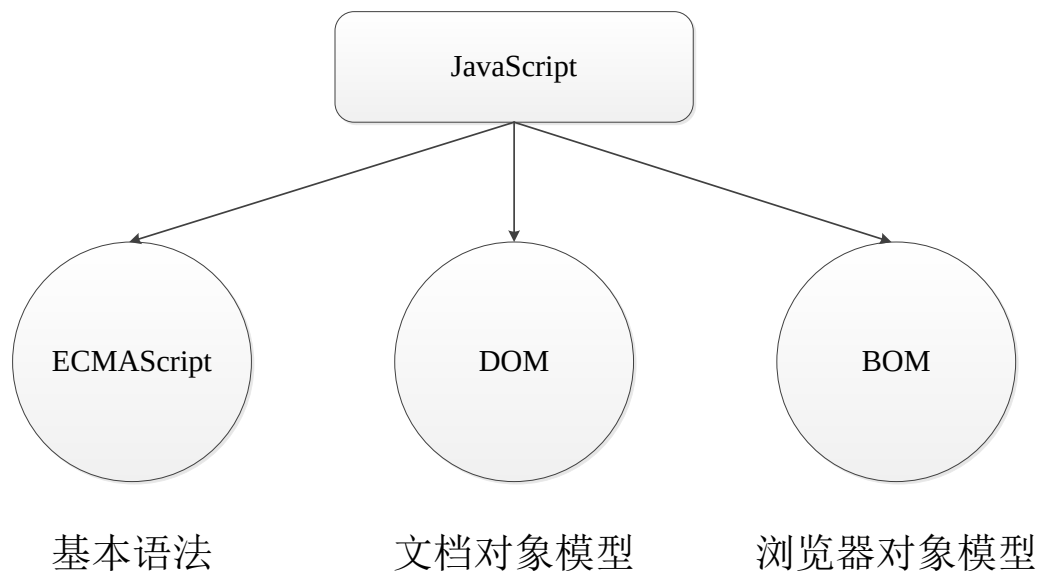


1.1 什么是JavaScript

4

JavaScript的组成

JavaScript由ECMAScript、DOM、BOM三部分组成。





1.1 什么是JavaScript

4

JavaScript的组成

JavaScript组成部分的简单介绍如下：

- ❑ ECMAScript: 是JavaScript的核心。它规定了JavaScript的编程语法和基础核心内容，是所有浏览器厂商共同遵守的一套JavaScript语法工业标准
- ❑ DOM: 文档对象模型，是W3C组织推荐的处理可扩展标记语言的标准编程接口，通过DOM提供的接口，可以对页面上的各种元素进行操作
- ❑ BOM: 浏览器对象模型，它提供了独立于内容的、可以与浏览器窗口进行互动的对象结构。通过BOM，可以对浏览器窗口进行操作



1.2 常用开发工具

工欲善其事，必先利其器，一款优秀的开发工具能够极大提高程序开发效率与体验。在Web前端开发中，常用的开发工具包括很多，例如：

- ❑ Visual Studio Code
- ❑ Sublime Text
- ❑ Hbuilder
- ❑ Adobe Dreamweaver
- ❑ WebStorm



1.3 JavaScript入门

1

代码书写位置

JavaScript代码的3种书写位置：

- ❑ 行内式：是将单行或少量的JavaScript代码写在HTML标签的事件属性中

```
<body>
```

```
<input type="button" value="点我" onclick="alert('行内式')" />
```

```
</body>
```

- ❑ 内嵌式（嵌入式）：使用<script>标签包裹JavaScript代码，<script>标签可以写在<head>或<body>标签中

```
<script type="text/javascript">
```

```
    alert('内嵌式')
```

```
</script>
```



1.3 JavaScript入门

1

代码书写位置

JavaScript代码的**3种**书写位置：

- 外部式（外链式）：将JavaScript代码写在一个单独的文件中，一般使用“js”作为文件的扩展名，在HTML页面中使用<script>标签进行引入，适合JavaScript代码量比较多的情况。注意外部式的<script>标签内不可以编写JavaScript代码

```
script src="test.js" type="text/javascript" charset="utf-8">  
</script>
```



1.3 JavaScript入门

1

代码书写位置

在编写JavaScript代码时，需要注意基本的语法规则：

- ❑ JavaScript**严格区分大小写**，在编写代码时一定要注意大小写的正确性
- ❑ JavaScript代码**对空格、换行、缩进不敏感**，一条语句可以分成多行书写
- ❑ 如果一条语句结束后，换行书写下一条语句，后面的分号可以省略



1.3 JavaScript入门

2

注释

JavaScript代码的注释方式，以及在VS Code编辑器中对应的快捷键如下：

- ❑ **单行注释**：以“//”开始，到该行结束或<script>标签结束之前的内容都是注释。快捷键：ctrl + /
- ❑ **多行注释**：以“/*”开始，以“*/”结束。需要注意的是，多行注释中可以嵌套单行注释，但不能再嵌套多行注释。快捷键：shift + alt + a



1.3 JavaScript入门

3

输入和输出语句

JavaScript代码中提供了输入和输出语句，可以在网页中实现用户交互效果。

常用的输入和输出语句如下所示：

- ❑ `alert(msg)`：浏览器弹出警告框
- ❑ `console.log(msg)`：浏览器控制台输出信息
- ❑ `prompt(msg)`：浏览器弹出输入框，用户可以输入内容



1.3 JavaScript入门

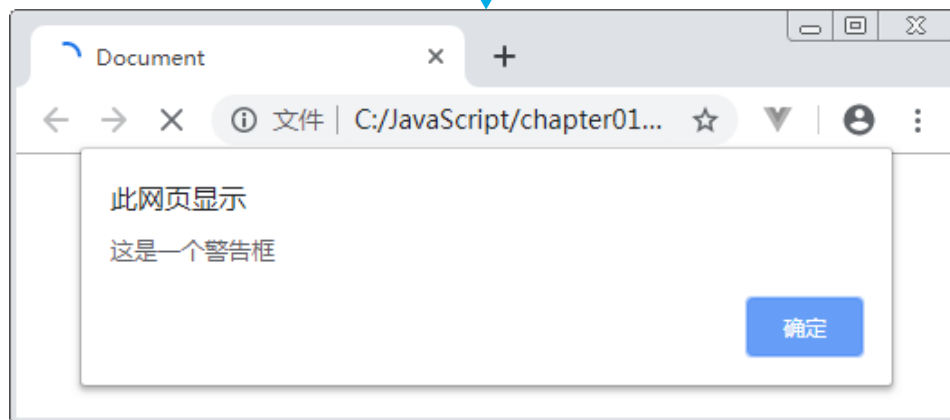
3

输入和输出语句

案例演示：alert()方法的使用。

示例代码

```
<script>  
  alert('这是一个警告框');  
</script>
```





1.3 JavaScript入门

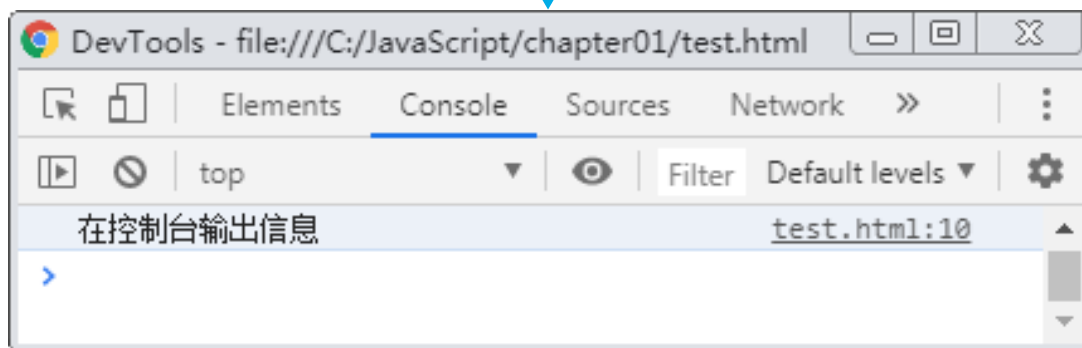
3

输入和输出语句

案例演示：console.log()方法的使用。

示例代码

```
<script>  
  console.log('在控制台输出信息');  
</script>
```





1.3 JavaScript入门

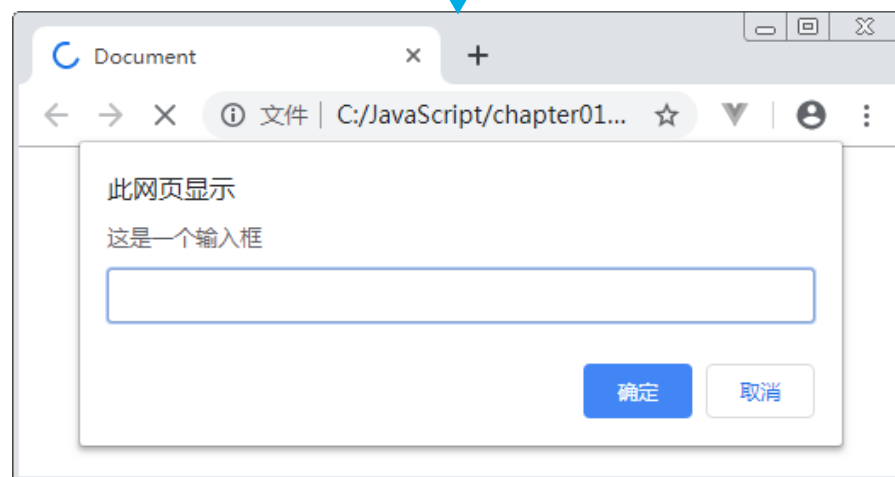
3

输入和输出语句

案例演示：prompt()方法的使用。

示例代码

```
<script>  
    prompt('这是一个输入框');  
</script>
```





1.3 JavaScript入门

4

控制台的使用

浏览器的控制台中也可以直接输入JavaScript代码中来执行语句，这为初学者提供了很大的便利。

用户输入

控制台输出

```
DevTools - file:///C:/JavaScript/chapter01/test.html  
Elements Console Sources Network >>  
top Filter Default levels  
var Obox = document.getElementById('box')  
Obox.innerHTML = '你好！'  
"你好！"  
> |
```

The screenshot shows the Chrome DevTools Console with the 'Console' tab selected. It displays two lines of JavaScript code that have been executed: `var Obox = document.getElementById('box')` and `Obox.innerHTML = '你好！'`. Below the code, the output `"你好！"` is shown. Red dashed boxes highlight the code lines, and blue arrows point from the '用户输入' (User Input) and '控制台输出' (Console Output) labels to these respective parts.

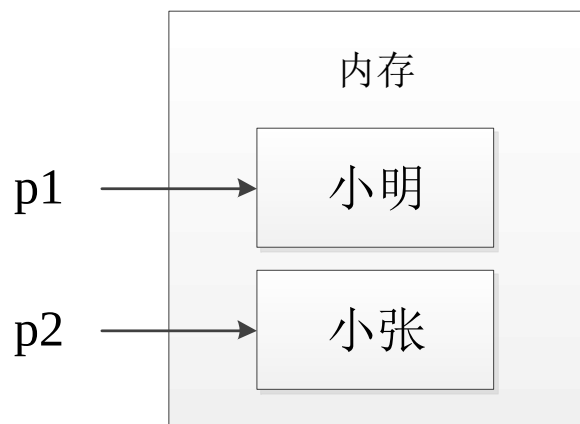


1.4 JavaScript变量

1

什么是变量

变量是程序在内存中申请的一块用来存放数据的空间。例如，程序在内存中保存字符串“小明”和“小张”，如下图所示。





1.4 JavaScript变量

2

变量的使用

变量的使用，分为两步：声明变量和变量赋值。

① 声明变量

```
var age;           // 声明一个名称为age的变量
```

② 变量赋值

```
age = 10           // 为age变量赋值
```

③ 变量初始化

```
var age = 18       // 声明变量的同时赋值
```



1.4 JavaScript变量

3

变量的应用案例

① 使用变量保存个人信息，示例代码如下：



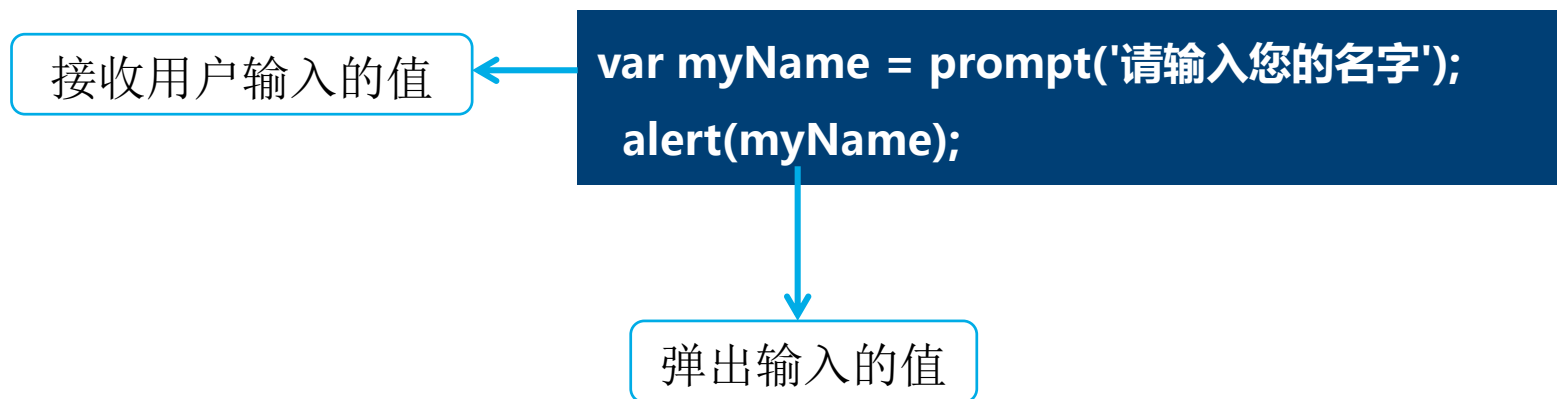


1.4 JavaScript变量

3

变量的应用案例

② 使用变量保存用户输入的值





1.4 JavaScript变量

4

变量的语法细节

① 更新变量的值，示例代码如下：

重新赋值

```
var myName = '小明';  
console.log(myName);    // 输出结果：小明  
myName = '小红';        // 更新变量的值  
console.log(myName);    // 输出结果：小红
```



1.4 JavaScript变量

4

变量的语法细节

② 同时声明多个变量

```
var myName, age, email;
```

```
var myName = '小明',
```

```
    age = 18,
```

```
    email = 'xiaoming@localhost';
```

同时声明多个变量不赋值

同时声明多个变量并赋值



1.4 JavaScript变量

4

变量的语法细节

③ 声明变量的特殊情况

□ 只声明变量，但不赋值。

undefined

```
var age;  
console.log(age);
```



1.4 JavaScript变量

4

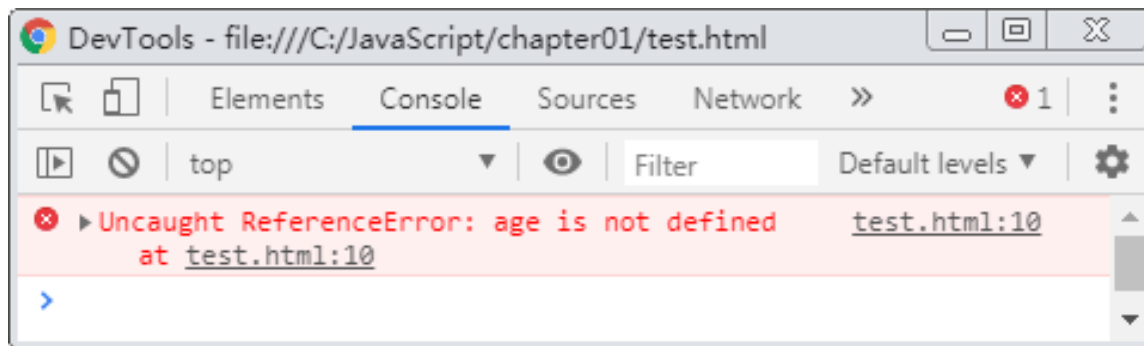
变量的语法细节

③ 声明变量的特殊情况

❑ 不声明变量，直接输出变量的值。

控制台会报错

`console.log(age);`



错误提示

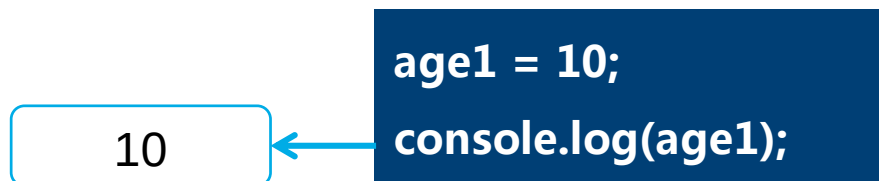


1.4 JavaScript变量

4

变量的语法细节

□ 不声明变量，只进行赋值。



注意：直接赋值一个未声明的变量，也可以正确输出变量的值。这个情况是JavaScript语言的特性。



1.4 JavaScript变量

5

变量的命名规范

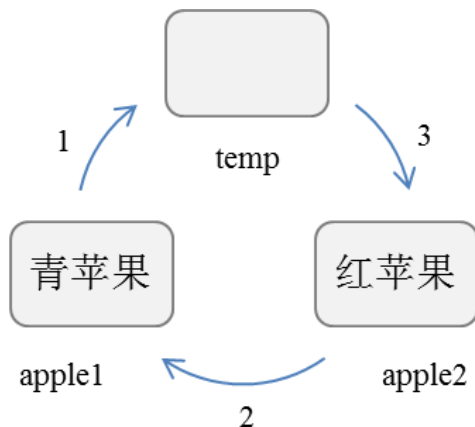
在对变量进行命名时，需要遵循变量的命名规范，具体如下：

- ❑ 由字母、数字、下划线和美元符号（\$）组成
- ❑ 严格区分大小写
- ❑ 不能以数字开头
- ❑ 不能是关键字、保留字
- ❑ 要尽量做到“见其名知其意”
- ❑ 建议遵循驼峰命名法，首字母小写，后面的单词首字母大写

1.4 JavaScript变量

6

【案例】交换两个变量的值



交换思想

示例代码

```
var temp;  
var apple1 = '青苹果';  
var apple2 = '红苹果';  
temp = apple1;  
apple1 = apple2;  
apple2 = temp;  
console.log(apple1); // 输出结果：红苹果  
console.log(apple2); // 输出结果：青苹果
```


本章总结

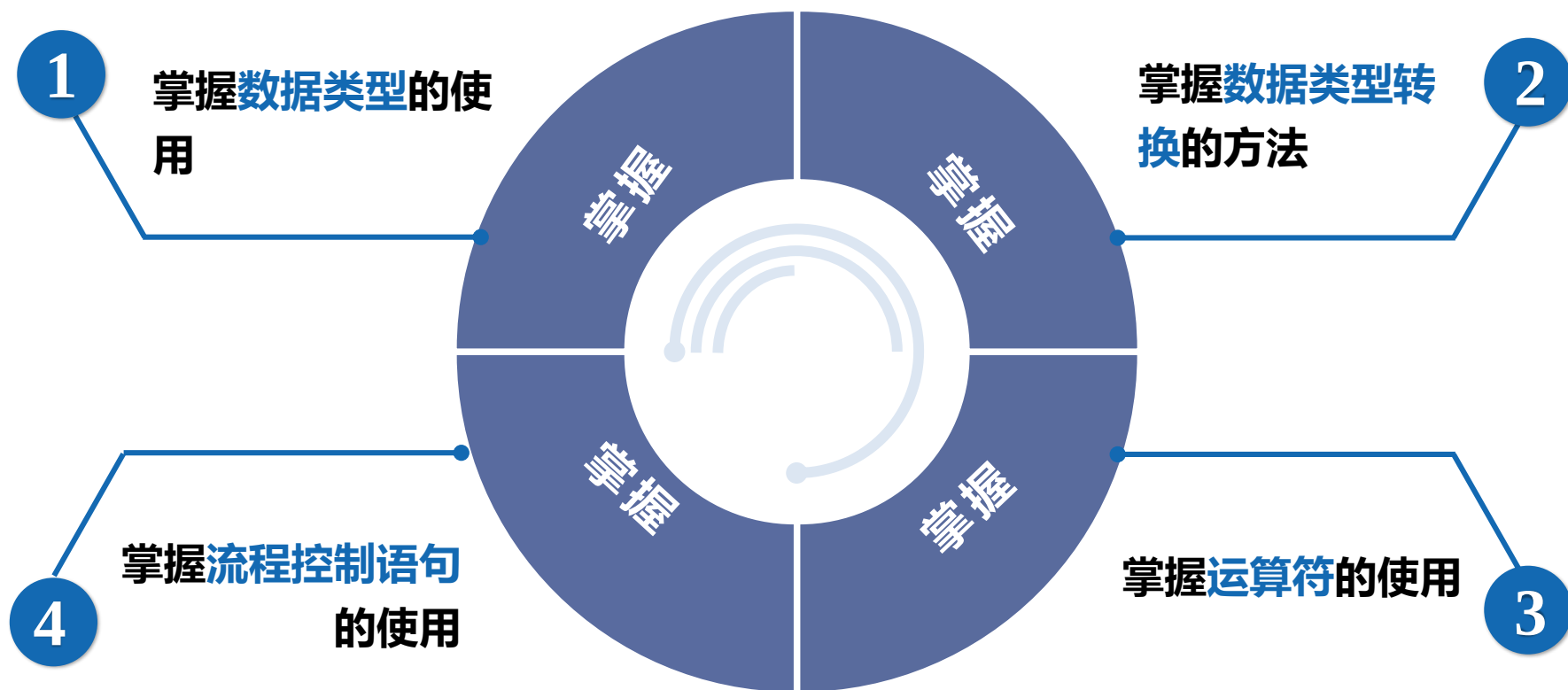


本章首先介绍了JavaScript的用途、发展状况，以及JavaScript的3大组成部分及其与ECMAScript的关系，然后讲解了常用开发工具相关的内容，接着针对JavaScript的入门知识进行了介绍，包括代码书写位置、注释、输入输出语句及控制台的使用，最后针对JavaScript变量进行了介绍，包括变量的本质、使用方式、语法规则及命名规范，最后通过案例来体验了JavaScript变量的简单应用。



第2章 JavaScript基础（上）

学习目标



目录

2.1

数据类型

👉 [点击查看本节相关知识点](#)

2.2

数据类型转换

👉 [点击查看本节相关知识点](#)

2.3

运算符

👉 [点击查看本节相关知识点](#)

目录

2.4

流程控制

 [点击查看本节相关知识点](#)

2.5

分支结构

 [点击查看本节相关知识点](#)



2.1 数据类型

1

变量的数据类型

JavaScript是一种**弱类型语言**，不用提前声明变量的数据类型。在程序运行过程中，变量的数据类型会被自动确定。

- 强类型语言：如：Java、C
- 弱类型语言：如：JavaScript

2.1 数据类型

2

数据类型分类

JavaScript中的数据类型分为两大类，基本数据类型和复杂数据类型（也称为引用数据类型）。

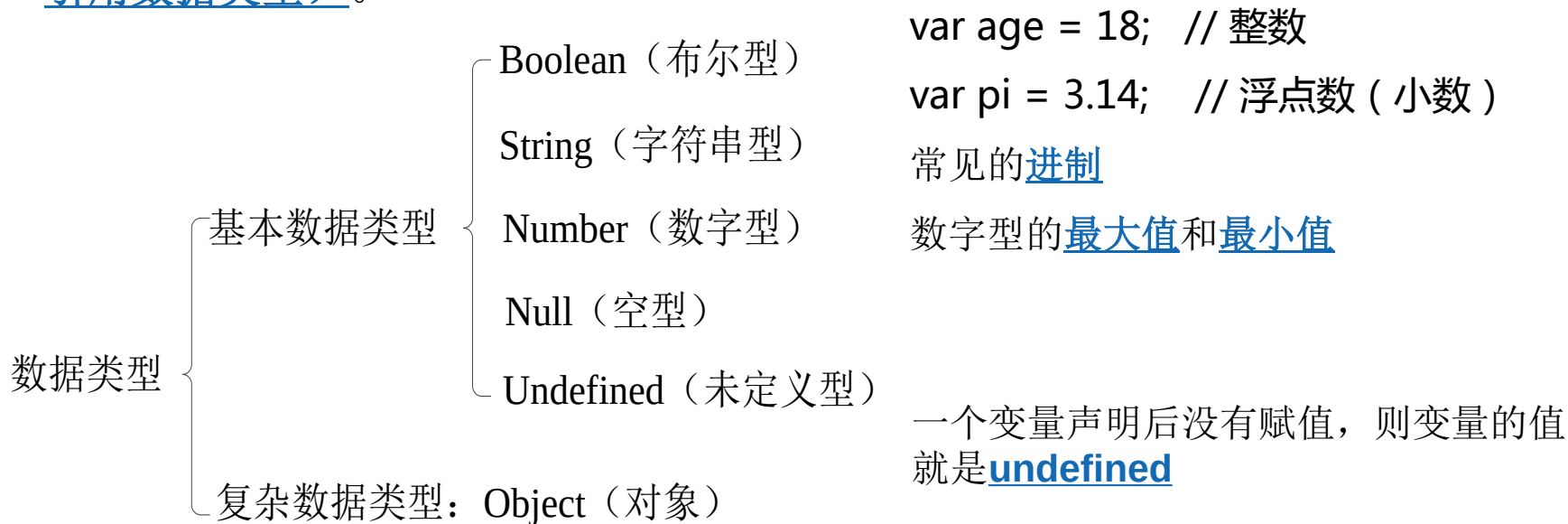


2.1 数据类型

2

数据类型分类

JavaScript中的数据类型分为两大类，基本数据类型和复杂数据类型（也称为引用数据类型）。



数据类型

typeof运算符进行数据类型检测，转PPT64页

2.1 数据类型

3

数字型

JavaScript中的数字型可以用来保存整数或浮点数（小数）。

示例代码

```
var age = 18; // 整数  
var pi = 3.14; // 浮点数（小数）
```

2.1 数据类型

3

数字型

① 常见的[进制](#):

- ❑ 八进制数: 在数字开头加上0, 八进制数由0~7组成, 逢8进位
- ❑ 十进制数: 是组成以10为基础的数字系统, 有0~9组成, 逢10进位
- ❑ 十六进制数: 在数字开头加上0x, 十六进制数由0~9, a~f组成

2.1 数据类型

3

数字型

② 范围：数字型的最大值和最小值：

❑ 最大值：Number.MAX_VALUE，输出结果：1.7976931348623157e+308

❑ 最小值：Number.MIN_VALUE，输出结果：5e-324

2.1 数据类型

3

数字型

③ 数字型的3个特殊值：Infinity和-Infinity和NaN，举例：

❑ Infinity（无穷大）：如`Number.MAX_VALUE*2`

❑ -Infinity（无穷小）：如`-Number.MAX_VALUE*2`

❑ NaN（非数值）：如`'abc' - 100`

❑ `isNaN`：用来判断一个变量是否为非数字的类型，返回值为`true`表示非数字，`false`表示是数字

2.1 数据类型

4

字符串型

字符串是指计算机中用于表示文本的一系列字符，在JavaScript中使用单引号或双引号来包裹字符串。

```
var str1 = '单引号字符串';  
var str2 = "双引号字符串";
```

2.1 数据类型

4

字符串型

① 单、双引号嵌套:

- ❑ 在单引号字符串中可以直接书写双引号
- ❑ 在双引号字符串中也可以直接书写单引号

2.1 数据类型

4

字符串型

示例代码

正确
语法

```
var str1 = 'I am a "programmer"; // I am a "programmer"  
var str2 = "I'm a 'programmer'"; // I'm a 'programmer'
```

错误
语法

```
var str1 = 'I'm a programmer'; // 单引号错误用法  
var str2 = "I'm a "programmer""; // 双引号错误用法  
var str3 = 'I am a programmer"; // 单双引号混用
```

2.1 数据类型

4

字符串型

② 转义符: 在字符串中使用换行、**Tab**等特殊符号时，可以用转义符来进行转义，转义符以“\”开始。常见转义符见下表。

转义符	解释说明	转义符	解释说明
\'	单引号	\"	双引号
\n	LF换行，n表示 newline	\v	跳格（Tab、水平）
\t	Tab符号	\r	CR换行
\f	换页	\\	反斜线（\）
\b	退格，b表示blank	\0	Null字节
\xhh	由2位十六进制数字 hh表示的ISO-8859-1字 符。如“\x61”表示“a”	\uhhhh	由4位十六进制数字hhhh表示的Unicode字 符。如“\u597d”表示“好”

2.1 数据类型

4

字符串型

- ③ 字符串长度：字符串是由若干字符组成的，这些字符的数量就是字符串的长度。通过字符串的`length`属性可以获取整个字符串的长度。

示例代码

```
var str1 = 'I\'m a programmer';  
console.log(str1.length);
```

获取str1的长度

2.1 数据类型

4

字符串型

④ 访问字符串中的字符：字符串可以使用 “[index]” 语法按照index（索引）

访问字符，index从0开始，一直到字符串的长度减1，如果超过了index最大值，会返回undefined。

示例代码

```
var str = 'I\'m a programmer';  
console.log(str[0]); // 输出结果：I  
console.log(str[1]); // 输出结果：'  
console.log(str[15]); // 输出结果：r  
console.log(str[16]); // 输出结果：undefined
```

2.1 数据类型

4

字符串型

- ⑤ 字符串拼接：多个字符串之间可以使用“+”进行拼接，如果数据类型不同，拼接前会把其他类型转成字符串，再拼接成一个新的字符串。

示例代码

```
var str = 'I\'m a programmer';  
console.log('a' + 'b');    // ab  
console.log('a' + 18);     // a18  
console.log('_' + true);   // _true  
console.log('12' + 14);    // 1214  
console.log(12 + 14);      // 两个数字相加，结果为26
```

2.1 数据类型

4

字符串型

- ⑥ “显示年龄”案例：弹出一个输入框，让用户输入年龄。输入后，单击“确定”按钮，程序弹出来一个警告框，显示内容为“您今年x岁了”，x表示刚才输入的年龄。

示例代码

```
var age = prompt('请输入您的年龄'); // 弹出一个输入框，让用户输入年龄
var msg = '您今年' + age + '岁了'; // 将年龄与输出的字符串拼接
alert(msg); // 弹出警告框，输出程序的处理结果
```

2.1 数据类型

5

布尔型

布尔型通常用于逻辑判断，它有两个值：true和false，表示事物的“真”和“假”。

示例代码

```
console.log(true);           // 输出结果：true
console.log(false);          // 输出结果：false
console.log(true + 1);        // 输出结果：2
console.log(false + 1);       // 输出结果：1
```

true会转换成1

false会转换成0

2.1 数据类型

6

undefined和null

如果一个变量声明后没有赋值，则变量的值就是[undefined](#)。下面通过代码演示undefined的使用。

示例代码

```
// 演示undefined的使用
```

```
console.log(true);           // 输出结果：true
```

```
var a;
```

```
console.log(a);              // 输出结果：undefined
```

```
console.log(a + '_');        // 输出结果：undefined_ (字符串型)
```

```
console.log(a + 1);          // 输出结果：NaN
```

2.1 数据类型

6

undefined和null

当然也可以给一个变量赋一个null值，下面通过代码演示null值的使用。

示例代码

```
// 演示null的使用
var b = null;
console.log(b + '_');      // 输出结果：null_ (字符串型)
console.log(b + 1);       // 输出结果：1 (b转换为0)
console.log(b + true);    // 输出结果：1 (b转换为0, true转换为1)
```

2.1 数据类型

7

数据类型检测

可以利用[`typeof`](#)运算符进行数据类型检测。

示例代码

```
console.log(typeof 12);           // 输出结果 : number
console.log(typeof null);         // 输出结果 : object
var a = '12';
console.log(typeof a == 'string'); // 输出结果 : true
```


2.1 数据类型

7

数据类型检测

值得一提

字面量，简单来说，就是用字面量来表示如何在代码中表达这个值，通过字面量，我们可以很容易地看出来它是哪种类型的值。

例如数字字面量：7、8；布尔字面量：true、false等。

2.2 数据类型转换

1

转换为字符串型

转换为字符串型的3种常见的方式：

- ❑ 利用“+”拼接字符串（最常用的一种方式）
- ❑ 利用toString()转换成字符串
- ❑ 利用String()转换成字符串

注意：null和undefined无法使用toString()方式进行转换；对于数字型的变量，可以在toString()的小括号中传入参数，来进行进制转换。

2.2 数据类型转换

2

转换为数字型

转换为数字型的4种常见的方式：

- ❑ 使用parseInt()将字符串转为整数
- ❑ 使用parseFloat()将字符串转为浮点数
- ❑ 使用Number()将字符串转为数字型
- ❑ 利用算术运算符（-、*、/）隐式转换

2.2 数据类型转换

2

转换为数字型

- ① “计算年龄”案例，要求在页面中弹出一个输入框，提示用户输入出生年份，利用出生年份计算用户的年龄。

示例代码

```
var year = prompt('请输入您的年龄');  
var age = 2020 - parseInt(year);  
alert('您今年已经' + age + '岁了');
```

year是字符串
需要进行转换

2.2 数据类型转换

2

转换为数字型

- ② “简单加法器”案例，要求在页面中弹出两个输入框，分别输入两个数字，然后返回两个数字相加的结果。

示例代码

```
var num1 = prompt('请输入第一个数 : ');  
var num2 = prompt('请输入第二个数 : ');  
var result = parseFloat(num1) +  
parseFloat(num2);  
alert('计算结果是 : ' + result);
```

使用parseFloat()将
字符串转为浮点数

2.2 数据类型转换

3

转换为布尔型

转换为布尔型使用 `Boolean()`，在转换时，代表空、否定的值会被转换为 `false`，如空字符串、0、NaN、null 和 undefined，其余的值转换为 `true`。

示例代码

```
console.log(Boolean(''));           // false
console.log(Boolean(0));             // false
console.log(Boolean(NaN));           // false
console.log(Boolean(null));          // false
console.log(Boolean(undefined));     // false
console.log(Boolean('小白'));       // true
console.log(Boolean(12));            // true
```

2.3 运算符

1

算术运算符

算术运算符用于对两个变量或值进行算术运算，与数学上的加、减、乘、除类似，常用的算术运算符如下。

运算符	运算	示例	解释说明
+	加	$1 + 5$	6
-	减	$8 - 4$	4
*	乘	$3 * 4$	12
/	除	$3 / 2$	1.5
%	取余数（取模）	$7 \% 5$	2

2.3 运算符

1

算术运算符

算术运算符的注意事项：

- ❑ 进行四则混合运算时，运算顺序要遵循数学中“先乘除后加减”的原则
- ❑ 在进行取模运算时，运算结果的正负取决于被模数（%左边的数）的符号，
与模数（%右边的数）的符号无关
- ❑ 在开发中尽量避免利用浮点数进行运算，因为有可能会因JavaScript的精度
导致结果的偏差
- ❑ 使用“+”和“-”可以表示正数或负数

2.3 运算符

1

算术运算符

值得一提

表达式是各种类型的数据、变量和运算符的集合，最简单的表达式可以是一个变量或字面量。

注意：表达式最终都会有一个返回值。

2.3 运算符

2

递增和递减运算符

使用递增（++）、递减（--）运算符可以快速地对变量的值进行递增和递减操作，它属于一元运算符，只对一个表达式进行操作。

- ❑ 前置递增（递减）运算符：递增和递减运算符写在变量前面，返回的是计算后的结果
- ❑ 后置递增（递减）运算符：递增和递减运算符写在变量后面，返回的是计算前的结果
- ❑ 递增和递减运算符的优先级高于“+”“-”等运算符

2.3 运算符

3

比较运算符

比较运算符用于对两个数据进行比较，其结果是一个布尔值，即true或false，常用的比较运算符及用法见下表。

运算符	运算	示例	结果
>	大于	5 > 5	false
<	小于	5 < 5	false
>=	大于或等于	5 >= 5	true
<=	小于或等于	5 <= 5	true
==	等于	5 == 4	false
!=	不等于	5 != 4	true
===	全等	5 === 5	true
!==	不全等	5 !== '5'	true

2.3 运算符

4

逻辑运算符

逻辑运算符用于对布尔值进行运算，其返回值也是布尔值。在实际开发中，逻辑运算符经常用于多个条件的判断，常用的逻辑运算符见下表。

运算符	运算	示例	结果
&&	与	a && b	a和b都为true，结果为true，否则为false
	或	a b	a和b中至少有一个为true，结果为true，否则为false
!	非	!a	若a为false，结果为true，否则相反

2.3 运算符

4

逻辑运算符

值得一提

位运算符，用来对数据进行二进制运算，将参与运算的操作数视为由二进制（0和1）组成的32位的串。

2.3 运算符

5

赋值运算符

赋值运算符用于将运算符右边的值赋给左边的变量，常用的逻辑运算符及示例如下。

示例代码

```
console.log(Boolean('')) // false
var age = 10;
age += 5;           // 相当于age = age + 5;
console.log(age);   // 输出结果 : 15
age -= 5;           // 相当于age = age - 5;
console.log(age);   // 输出结果 : 10
age *= 10;          // 相当于age = age * 10;
console.log(age);   // 输出结果 : 100
```

2.3 运算符

5

赋值运算符

常用的逻辑运算符及示例见下表。

运算符	运算	示例	结果
=	赋值	a = 3;	a = 3
+=	加并赋值	a = 3; a += 2;	a = 5
-=	减并赋值	a = 3; a -= 2;	a = 1
*=	乘并赋值	a = 3; a *= 2;	a = 6
/=	除并赋值	a = 3; a /= 2;	a = 1.5
%=	模并赋值	a = 3; a %= 2;	a = 1
+=	连接并赋值	a = 'abc'; a += 'def';	a = 'abcdef'
<<=	左移位并赋值	a = 9; a <<= 2;	a = 36
>>=	右移位并赋值	a = -9; a >>= 2;	a = -3
>>>=	无符号右移位并赋值	a = 9; a >>>= 2;	a = 2
&=	按位“与”并赋值	a = 3; a &= 9;	a = 1
^=	按位“异或”并赋值	a = 3; a ^= 9;	a = 10
=	按位“或”并赋值	a = 3; a = 9;	a = 11

2.3 运算符

6

三元运算符

三元运算符是一种需要三个操作数的运算符，运算的结果根据给定条件决定。

语法格式

条件表达式 ? 表达式1 : 表达式2

语法说明：先求条件表达式的值，如果为true，则返回表达式1的执行结果；如果条件表达式的值为false，则返回表达式2的执行结果。

2.3 运算符

7

运算符优先级

运算符优先级：是指表达式中所有运算符参与运算的先后顺序，优先级如下表。

结合方向	运算符	结合方向	运算符
无	()	左	== != === !==
左	. [] new（有参数，无结合性）	左	&
右	new（无参数）	左	^
无	++（后置） --（后置）	左	
右	! ~ -（负数） +（正数） ++ （前置） --（前置） typeof void delete	左	&&
右	**	左	
左	* / %	右	?:
左	+ -	右	= += = *= /= %= <<= >>= >>>= &= ^= =
左	<< >> >>>	左	,
左	< <= > >= in instanceof		



2.4 流程控制

流程控制：通过控制代码的执行顺序来完成要实现的功能，流程控制的3种结构如下：

- ❑ **顺序**结构是程序中最基本的结构，程序会按照代码的先后顺序依次执行
- ❑ **分支**结构用于根据条件来决定是否执行某个分支代码
- ❑ **循环**结构用于根据条件来决定是否重复执行某一段代码

2.5 分支结构

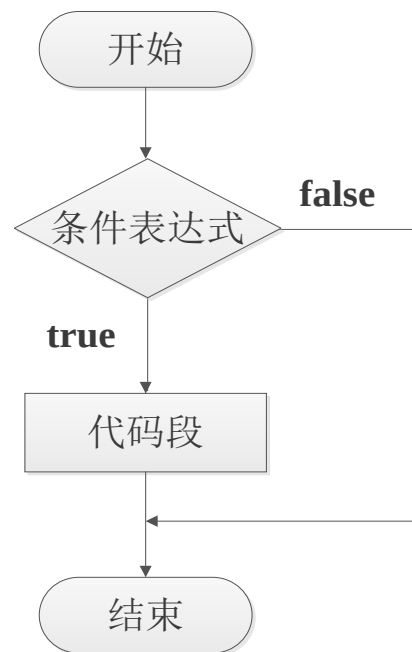
1

if语句

if语句也称为条件语句、单分支语句，当满足某种条件时，就进行某种处理。

```
if ( 条件表达式 ) {  
    // 代码段  
}
```

语法结构



流程图

2.5 分支结构

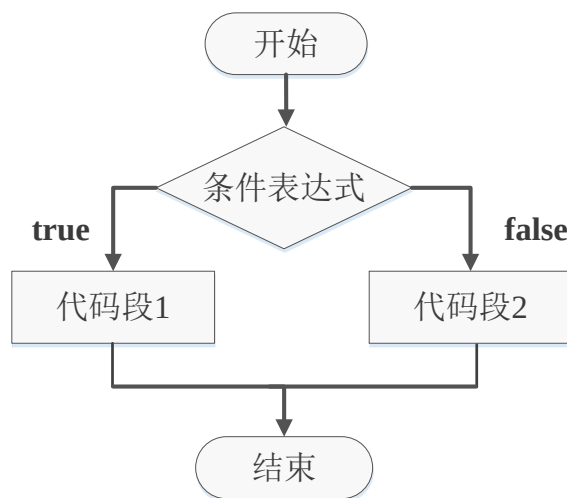
2

if...else语句

if...else语句也称为双分支语句，当满足某种条件时，就进行某种处理，否则进行另一种处理。

```
if ( 条件表达式 ) {  
    // 代码段1  
} else {  
    // 代码段2  
}
```

语法结构



流程图

2.5 分支结构

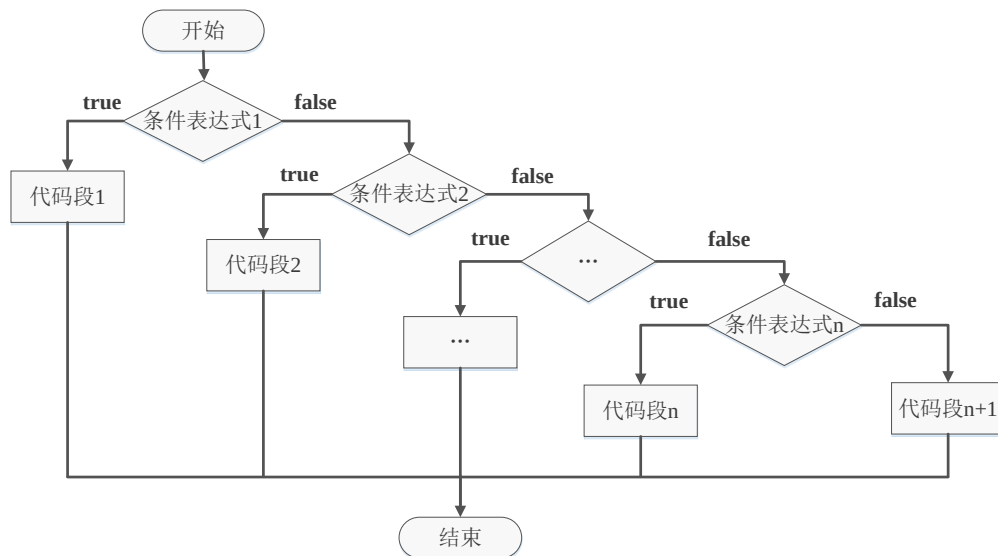
3

if...else if语句

if...else if语句也称为多分支语句，可针对不同情况进行不同的处理。

语法结构

```
if ( 条件表达式1 ) { // 代码段1 }  
else if ( 条件表达式2 ) { // 代码段2 }  
...  
else if ( 条件表达式n ) { // 代码段n }  
else { // 代码段n+1 }
```



流程图

2.5 分支结构

4

switch语句

switch语句也是[多分支语句](#)，功能与if...else if语句类似，不同的是它只能针对某个表达式的值作出判断，从而决定执行哪一段代码。

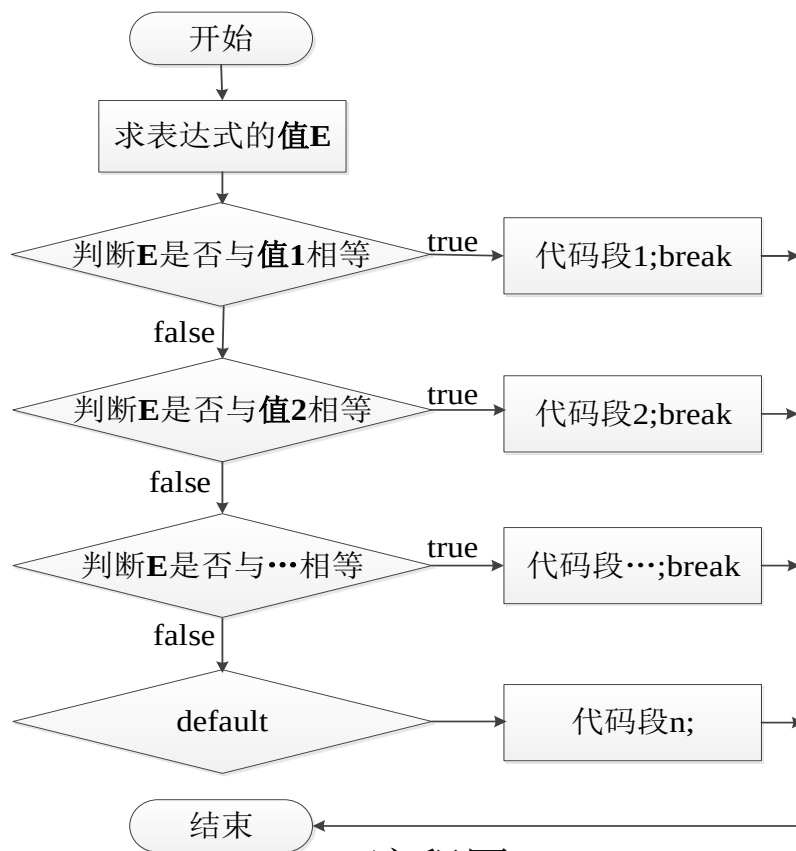
语法结构

```
switch ( 表达式 ) {  
    case 值1  
        代码段1;  
        break;  
    case 值2  
        代码段2;  
        break;  
    ...  
    default:  
        代码段n;  
}
```

2.5 分支结构

4

switch语句



流程图

2.5 分支结构

5

【案例】查询水果的价格

案例需求: 使用多分支语句实现用户在弹出框中输入一个水果，如果有就弹出该水果的价格，如果该水果不存在就弹出“没有此水果”的效果。

```
var fruit = prompt('请您输入查询的水果 : ');
switch(fruit){
  case '苹果':
    alert('苹果的价格是 3.5/斤');
    break;
  case '榴莲':
    alert('榴莲的价格是 35/斤');
    break;
  case '香梨':
    alert('香梨的价格是 3/斤');
    break;
  default:
    alert('没有此水果')
}
```



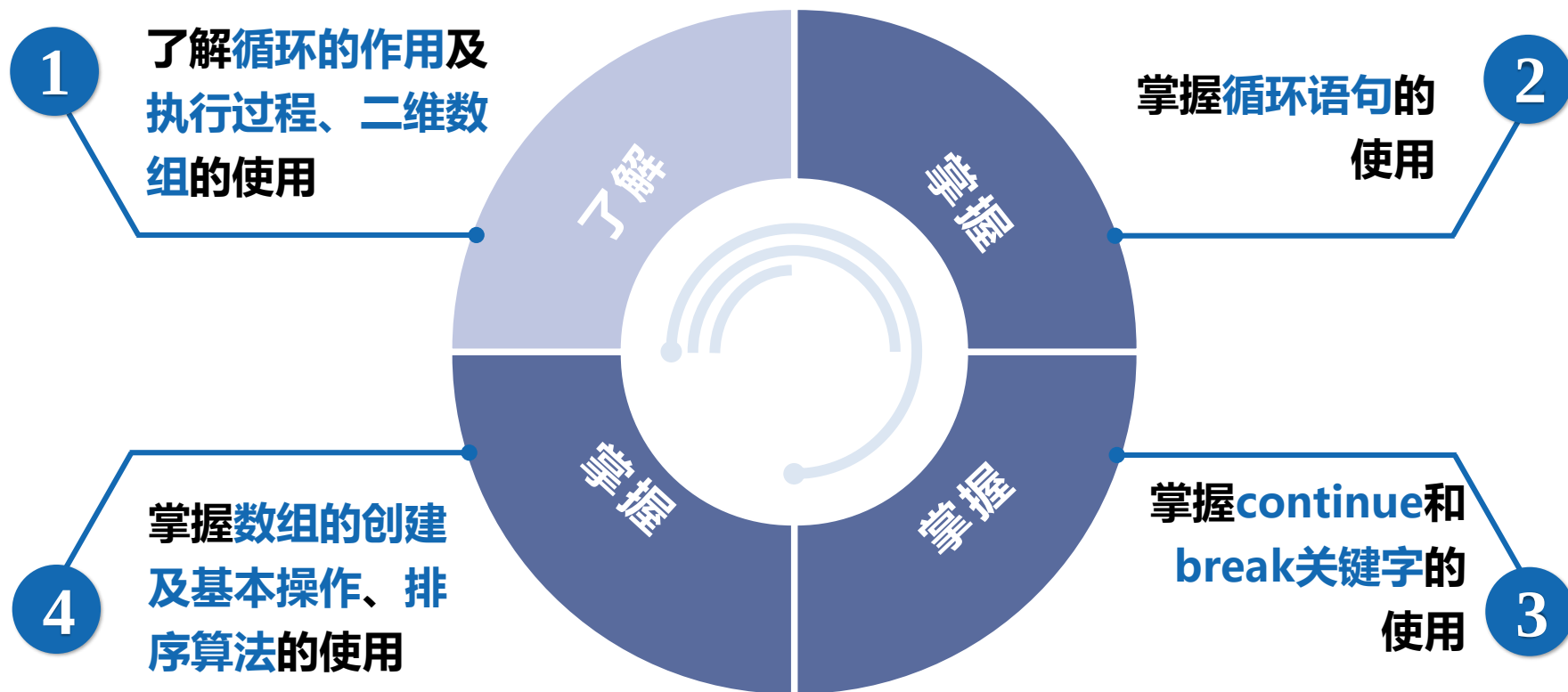

本章总结

本章首先介绍JavaScript中最基础的语法知识，包括数据类型的概念及数据类型的转换、运算符的使用，然后讲解了如何使用流程控制语句实现条件判断和代码的重复执行，最后以案例的形式讲解了多分支语句的实际应用，使程序变得更加的灵活。



第3章 JavaScript基础（下）

学习目标



目录

3.1

循环语句

 [点击查看本节相关知识点](#)

3.2

初识数组

 [点击查看本节相关知识点](#)

3.3

数组案例

 [点击查看本节相关知识点](#)

3.4

数组元素操作

 [点击查看本节相关知识点](#)

目录



数组排序算法

 [点击查看本节相关知识点](#)



二维数组

 [点击查看本节相关知识点](#)

3.1 循环结构

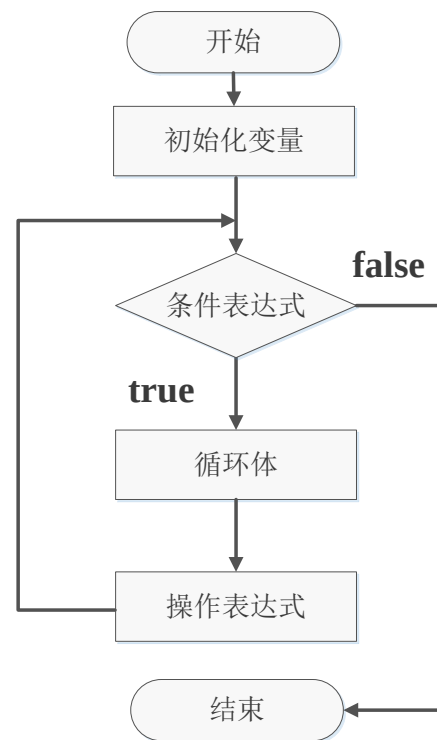
1

for语句

for语句是最常用的循环语句，它适合循环次数已知的情況。

```
for (初始化变量; 条件表达式; 操作表达式) {  
    // 循环体  
}
```

语法结构



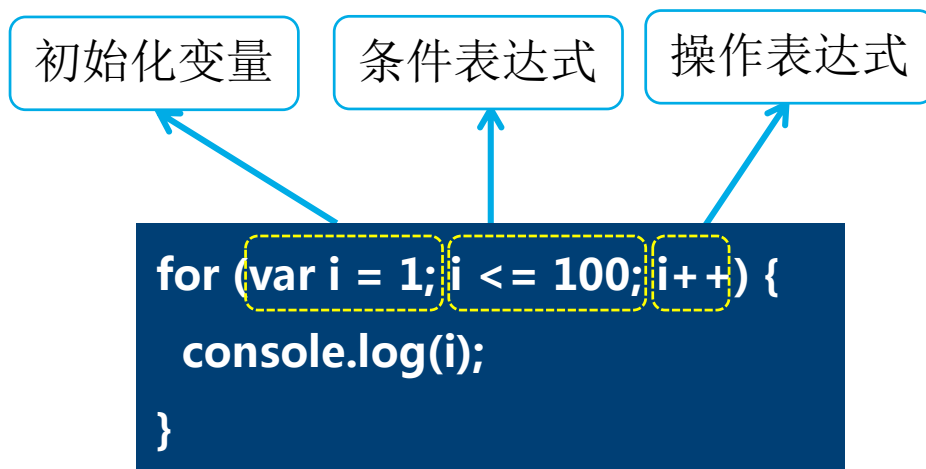
流程图

3.1 循环结构

1

for语句

案例： 使用for语句输出1~100范围内的数字



3.1 循环结构

1

for语句

代码执行流程:

- ❑ 执行 “`var i = 1;`” 初始化变量
- ❑ 判断 “`i <= 100`” 是否为true，如果为true，执行循环体，反之，结束循环
- ❑ 执行循环体，通过 “`console.log(i)`” 输出变量i的值
- ❑ 执行 “`i++`”，将i的值加1，此时i的值为2
- ❑ 判断 “`i <= 100`” 是否为true，和第②步相同。只要满足 “`i <= 100`” 这个条件，就会一直循环。当i的值加到101时，判断结果为false，循环结束



3.1 循环结构

1

for语句

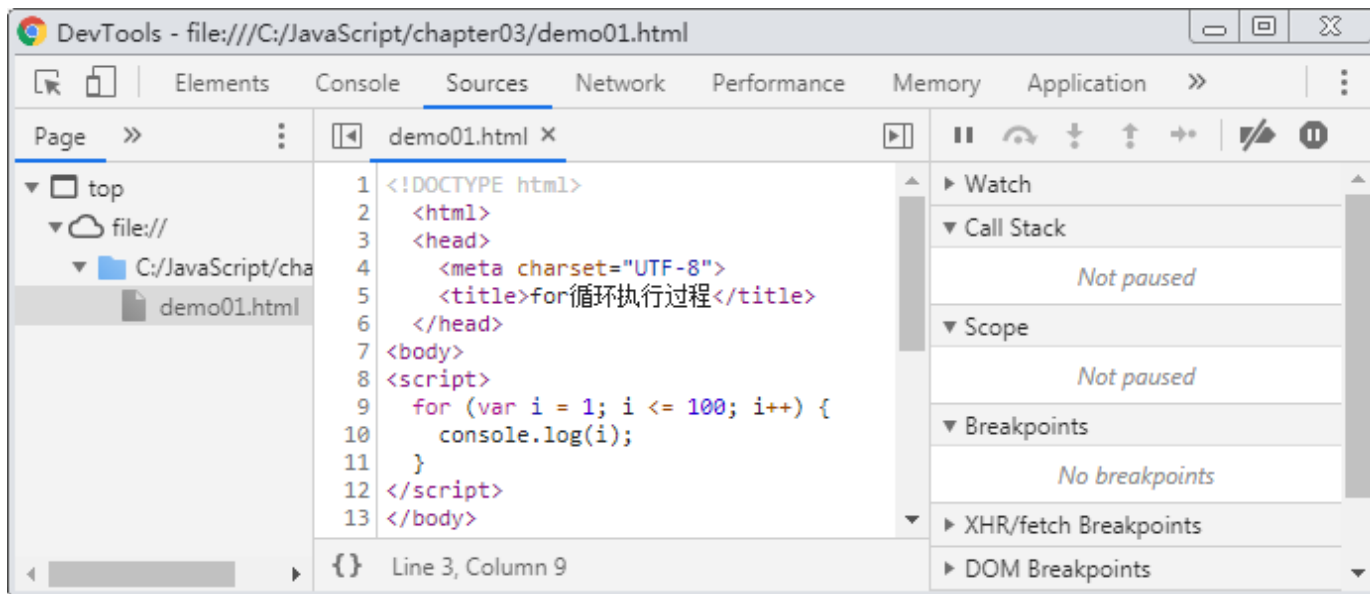
断点调试是指在程序的某一行设置一个断点，调试时，程序运行到这一行就会停住，然后就可以控制代码一步一步的执行，在这个过程中可以看到每个变量当前的值。

3.1 循环结构

1

for语句

查看方式: 在Chrome浏览器中，按F12键启动开发者工具后，切换到“Sources”面板，界面效果如下图所示，其中右栏是JavaScript调试区。



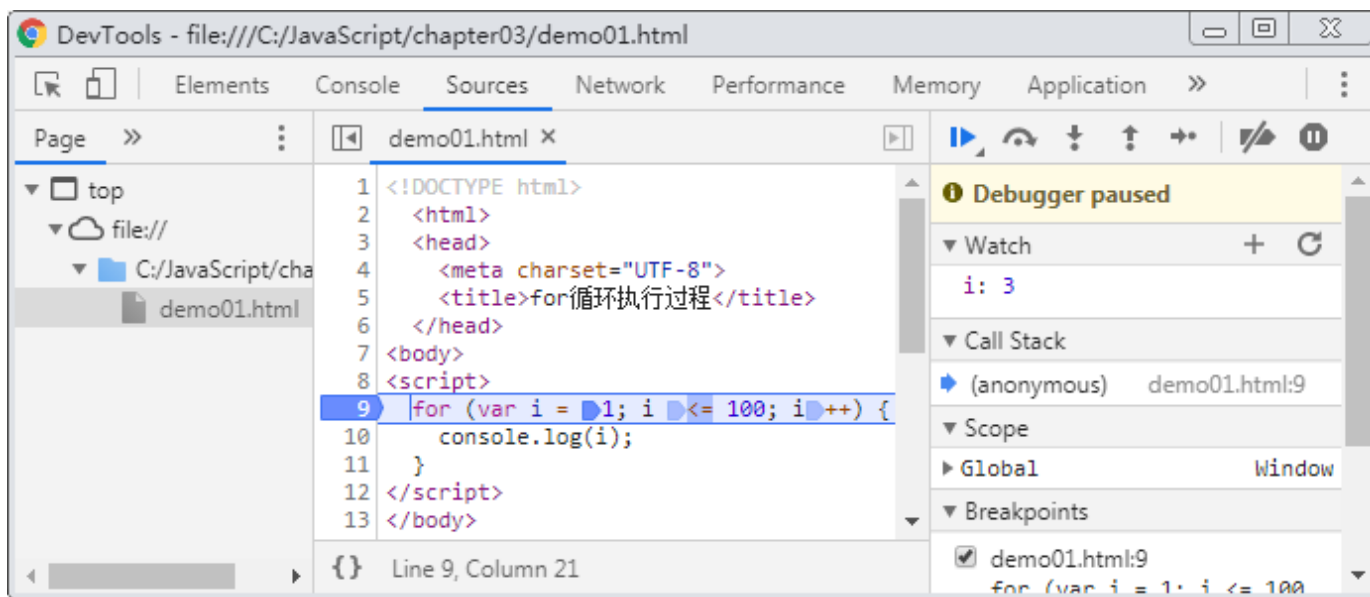
“Sources” 面板

3.1 循环结构

1

for语句

在中栏显示的网页源代码中，单击某一行的行号，即可添加断点，再次单击，可以取消断点。例如，为for语句添加断点，如下图所示。



断点调试

3.1 循环结构

2

for循环案例

① “重复执行相同代码”案例，重复执行N次。

```
var num = prompt('请您输入次数');  
for (var i = 1; i <= num; i++) {  
    console.log('重要的事情说' + num + '遍');  
}
```

3.1 循环结构

2

for循环案例

② “重复执行不同代码” 案例

```
for (var i = 1; i <= 100; i++) {  
  if (i == 1) { // 使用if条件判断语句，根据i的不同，进行不同处理  
    console.log('当前是第1次');  
  } else if (i == 100) {  
    console.log('当前是第100次');  
  }  
}
```

3.1 循环结构

2

for循环案例

③ 1~100之间的所有整数“求和”和“平均数”案例

```
var sum = 0;      // 利用sum对计数器i进行累加
for (var i = 1; i <= 100 ; i++) {
    sum += i;      // 相当于sum = sum + i;
}
console.log('求和 : ' + sum);           // 计算结果 : 5050
console.log('求平均值 : ' + (sum / 100)); // 计算结果 : 50.5
```

3.1 循环结构

2

for循环案例

④ 1~100之间的所有整数“求偶数和”和“求奇数和”案例

```
var even = 0;
var odd = 0;
for (var i = 1; i <= 100; i++) {
  if (i % 2 == 0) { // 判断i是奇数还是偶数
    even += i;
  } else { odd += i; }
}
console.log('1~100之间所有的偶数和是' + even); // 计算结果 : 2550
console.log('1~100之间所有的奇数和是' + odd); // 计算结果 : 2500
```

3.1 循环结构

2

for循环案例

⑤ 1~100之间的所有能被3整除的整数之和案例

```
var result = 0;
for (var i = 1; i <= 100; i++) {
  if (i % 3 == 0) {
    result += i;
  }
}
console.log(result);           // 计算结果 : 1683
```


3.1 循环结构

2

for循环案例

⑥ 自动生成字符串案例

```
var num = prompt('请输入星星的个数');  
var str = '';  
for (var i = 1; i <= num; i++) {  
    str = str + '★';  
}  
console.log(str);
```

3.1 循环结构

3

循环嵌套案例

① 生成i行j列的星星图案案例

```
var rows = prompt('请输入行数 : ');
var cols = prompt('请输入列数 : ');
var str = '';
for (var i = 1; i <= rows; i++) {
  for (var j = 1; j <= cols; j++) {
    str += '☆';
  }
  str += '\n';      // 换到下一行
}
console.log(str);
```



星星图案

3.1 循环结构

3

循环嵌套案例

② 生成三角形的星星图案案例

```
var str = "";
for (var i = 1; i <= 5; i++) {
  for (var j = i; j <= 5; j++) {
    // j的初始值为i
    str = str + '☆';
  }
  str += '\n';
}
console.log(str);
```



三角形图案

3.1 循环结构

3

循环嵌套案例

③ 生成九九乘法表案例

```
var str = '';
for (var i = 1; i <= 9; i++) {
  for (var j = 1; j <= i; j++) {
    str += j + 'x' + i + '=' + i * j + '\t';
  }
  str += '\n';
}
console.log(str);
```

```
1x1=1
1x2=2 2x2=4
1x3=3 2x3=6 3x3=9
1x4=4 2x4=8 3x4=12 4x4=16
1x5=5 2x5=10 3x5=15 4x5=20 5x5=25
1x6=6 2x6=12 3x6=18 4x6=24 5x6=30 6x6=36
1x7=7 2x7=14 3x7=21 4x7=28 5x7=35 6x7=42 7x7=49
1x8=8 2x8=16 3x8=24 4x8=32 5x8=40 6x8=48 7x8=56 8x8=64
1x9=9 2x9=18 3x9=27 4x9=36 5x9=45 6x9=54 7x9=63 8x9=72 9x9=81
```

九九乘法表

3.1 循环结构

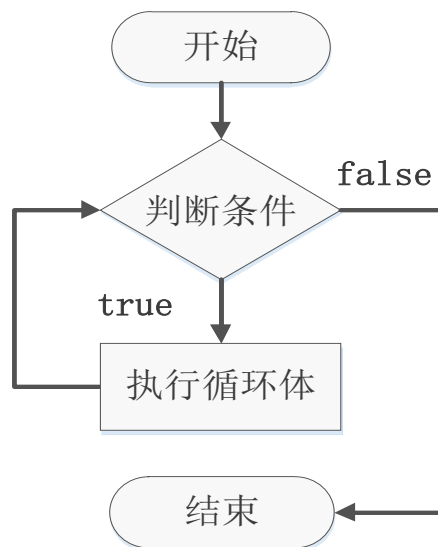
4

while语句

while语句可以在条件表达式为true的前提下，循环执行指定的一段代码，直到条件表达式为false时结束循环。

```
while (条件表达式) {  
    // 循环体  
}
```

语法结构



流程图

3.1 循环结构

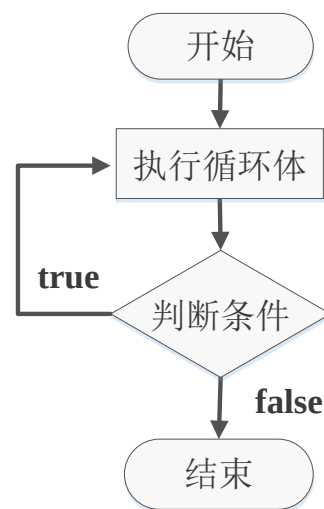
5

do...while语句

do...while语句会无条件地执行一次循环体中的代码，然后再判断条件，根据条件决定是否循环执行。

```
do {  
    // 循环体  
} while(条件表达式)
```

语法结构



流程图

3.1 循环结构

6

continue关键字

continue关键字可以在for、while以及do...while循环体中使用，它用来立即跳出本次循环，也就是跳过了continue后面的代码，继续下一次循环。

3.1 循环结构

7

break关键字

break关键字的用法：

- ❑ 用在switch语句中，当遇到break语句时，跳出switch语句
- ❑ 循环语句中使用时，其作用是立即跳出整个循环（将循环结束）

注意：break语句还可跳转到指定的标签语句处，实现循环嵌套中的多层跳转

3.2 初识数组

1

创建数组

创建数组的两种常见方式的用法：

- 使用 “new Array()” 创建数组
- 使用 “[]” 字面量来创建数组

```
// 使用new Array()创建数组  
var arr1 = new Array();  
// 使用字面量来创建数组  
var arr1 = [];
```

3.2 初识数组

2

访问数组元素

使用[索引](#)来访问数组中的元素，索引是一个数字，从0开始。

```
var arr = ['苹果', '橘子', '香蕉', '桃子'];
```

索引： ↓ ↓ ↓ ↓
 0 1 2 3

```
var arr = ['苹果', '橘子', '香蕉', '桃子'];  
console.log(arr[0]);        // 输出结果：苹果  
console.log(arr[1]);        // 输出结果：橘子  
console.log(arr[2]);        // 输出结果：香蕉  
console.log(arr[3]);        // 输出结果：桃子  
console.log(arr[4]);  
// 输出结果：undefined ( 数组元素不存在 )
```



3.2 初识数组

3

数组遍历

数组遍历是将数组中的元素全部访问一遍，可以利用for循环来实现，在for循环中让索引从0开始自增。

如果数组元素比较多时，计算数组元素的个数不太方便，这时候可以利用“数组名.length”来快速地获取数组长度。

3.3 数组案例

1

获取数组元素中的最大值

案例思路：数组的遍历可以获取数组中的最大值。在遍历时，先用一个变量max保存数组中第1个元素的值，然后比较后面的元素是否比max的值大，如果比max大，就将这个较大的值保存给max，否则就进行下一轮的比较。

```
var arr = [2, 6, 1, 77, 52, 25, 7, 99];  
var max = arr[0];  
for (var i = 1; i < arr.length; i++) {  
    if (arr[i] > max) {    max = arr[i]; }  
}  
console.log('数组元素中的最大值是：' + max); // 计算结果：99
```

3.3 数组案例

2

数组转换为字符串

案例需求: 将数组 “['red', 'green', 'blue', 'pink']” 转换为字符串，并用 “|” 或其他符号来分隔每个元素，如 “red|green|blue|pink”。

```
var arr = ['red', 'green', 'blue', 'pink'];  
var str = arr[0];  
var sep = '|';  
for (var i = 1; i < arr.length; i++) {  
    str += sep + arr[i];  
}  
console.log(str); // 输出结果 : red|green|blue|pink
```

3.4 数组元素操作

1

修改数组长度

使用“数组名.length”可以获取或修改数组的长度。

① 获取数组长度

```
var arr = ['a', 'b', 'c'];  
console.log(arr.length); // 输出结果：3
```



数组长度=数组索引+1

3.4 数组元素操作

1

修改数组长度

② 修改数组长度

```
var arr1 = [1, 2];  
arr1.length = 4;           // 大于原有长度  
console.log(arr1);           // 输出结果 : (4) [1, 2, empty × 2]  
var arr2 = [1, 2, 3, 4];  
arr2.length = 2;             // 小于原有长度  
console.log(arr2);           // 输出结果 : (2) [1, 2]
```

3.4 数组元素操作

1

修改数组长度

③ 访问空元素时，返回结果为undefined

```
var arr1 = [1, 2];  
var arr = [1];  
arr.length = 4;           // 修改数组的长度为4  
console.log(arr);          // 输出结果 : (4) [1, empty × 3]  
console.log(arr[1]);        // 输出结果 : undefined
```


3.4 数组元素操作

1

修改数组长度

出现空元素的其他常见的情况：

```
// 情况1：在使用字面量创建数组时出现空元素
var arr = [1, 2, , 4];
console.log(arr);           // 输出结果：(4) [1, 2, empty, 4]

// 情况2：在new Array()中传入数组长度的参数
var arr = new Array(4);
console.log(arr);           // 输出结果：(4) [empty × 4]

// 情况3：为数组添加索引不连续的元素
var arr = [1];
arr[3] = 4;                  // 向数组中添加一个元素，索引为3
console.log(arr);           // 输出结果：(4) [1, empty × 2, 4]
```

3.4 数组元素操作

2 新增或修改数组元素

如果给定的索引超过了数组中的最大索引，则表示新增元素，否则表示修改元素。

```
var arr = ['red', 'green', 'blue'];  
arr[3] = 'pink';      // 新增元素  
console.log(arr);     // (4) ["red", "green", "blue", "pink"]  
arr[0] = 'yellow';    // 修改元素  
console.log(arr);     // (4) ["yellow", "green", "blue", "pink"]
```

3.4 数组元素操作

3

筛选数组元素

案例需求: 将一个数组中所有大于或等于10的元素筛选出来，放入到新的数组中。

```
var arr = [2, 0, 6, 1, 77, 0, 52, 0, 25, 7];
var newArr = [];
var j = 0;
for (var i = 0; i < arr.length; i++) {
  if (arr[i] >= 10) {
    newArr[j++] = arr[i]; // 新数组索引号从0开始，依次递增
  }
}
console.log(newArr); // 输出结果：(3) [77, 52, 25]
```

3.4 数组元素操作

4

删除指定的数组元素

案例需求: 将一个数组中所有数值为0的元素删除。

```
var arr = [2, 0, 6, 1, 77, 0, 52, 0, 25, 7];  
var newArr = [];  
for (var i = 0; i < arr.length; i++) {  
    if (arr[i] !== 0) {  
        newArr[newArr.length] = arr[i];  
    }  
}  
console.log(newArr);           // 输出结果 : (7) [2, 6, 1, 77, 52, 25, 7]
```

3.4 数组元素操作

5

反转数组元素顺序

案例需求: 将一个数组中所有元素的顺序反过来。

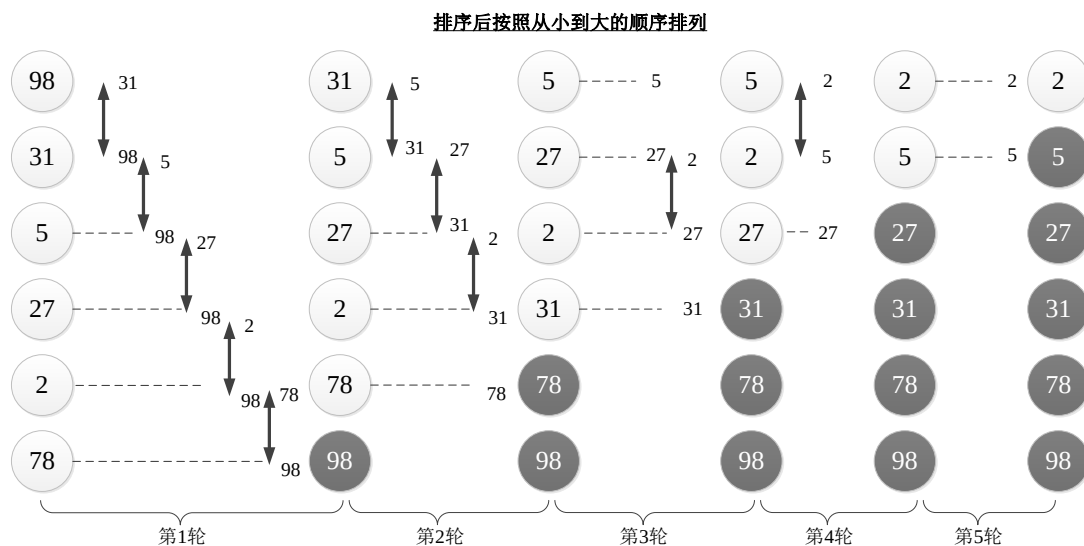
```
var arr = ['red', 'green', 'blue', 'pink', 'purple'];  
var newArr = [];  
for (var i = arr.length - 1; i >= 0; i--) {  
    newArr[newArr.length] = arr[i];  
}  
// 输出结果 : (5) ["purple", "pink", "blue", "green", "red"]  
console.log(newArr);
```

3.5 数组排序算法（自学）

1

冒泡排序

冒泡排序算法：在冒泡排序的过程中，按照要求从小到大排序或从大到小排序，不断比较数组中相邻两个元素的值，较小或较大的元素前移。



冒泡排序

3.5 数组排序算法

1

冒泡排序

案例需求：实现从小到大排序。

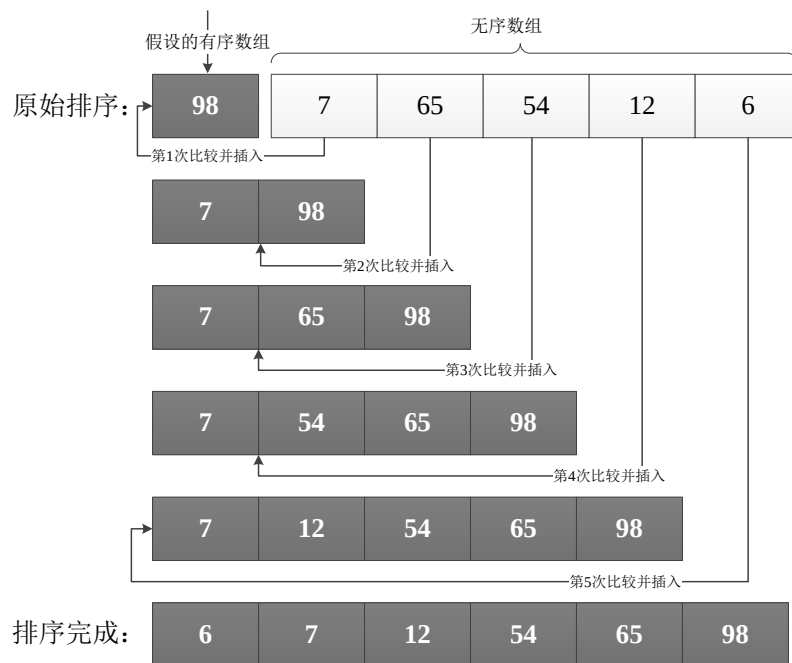
```
var arr = [10, 7, 5, 27, 98, 31];           // 待排序数组
for (var i = 1; i < arr.length; i++) {      // 控制需要比较的轮数
  for (var j = 0; j < arr.length - i; j++) { // 控制参与比较的元素
    if (arr[j] > arr[j + 1]) {               // 比较相邻的两个元素
      var temp = arr[j + 1];
      arr[j + 1] = arr[j];
      arr[j] = temp;
    }
  }
}
console.log(arr); // 输出结果 : 5,7,10,27,31,98
```

3.5 数组排序算法

2

插入排序

插入排序算法：通过构建有序数组元素的存储，对未排序的数组元素，在已排序的数组中从最后一个元素向第一个元素遍历，找到相应位置并插入。



从小到大插入排序

3.5 数组排序算法

2

插入排序

案例需求：实现从小到大排序。

```
var arr = [10, 8, 100, 31, 87, 70, 1, 88];  
// 按照从小到大的顺序排列，先遍历无序数组下标  
for (var i = 1; i < arr.length; i++) {  
  //遍历并比较一个无序数组元素与所有有序数组元素  
  for (var j = i; j > 0; j--) {if (arr[j - 1] > arr[j]) {  
    var temp = arr[j - 1];  
    arr[j - 1] = arr[j];  
    arr[j] = temp;  
  }}  
}  
console.log(arr); // 输出结果：1,8,10,31,70,87,88,100
```

3.6 二维数组

1

创建二维数组

创建二维数组的两种常见方式的用法：

① 使用 “new Array()” 字面量来创建数组

```
var info = new Array(  
  new Array('Tom', 13, 155),  
  new Array('Lucy', 11, 152)  
);  
console.log(info[0]);      // 输出结果 : (3) ["Tom", 13, 155]  
console.log(info[0][0]);   // 输出结果 : Tom
```

3.6 二维数组

1

创建二维数组

创建二维数组的两种常见方式的用法：

② 使用 “[]” 字面量来创建数组

```
var nums = [[1, 2], [3, 4]];
console.log(nums[0]);    // 输出结果：(2) [1, 2]
console.log(nums[1][0]); // 输出结果：3
```

3.6 二维数组

2

二维数组求和

二维数组求和是对二维数组进行遍历，需要用**双层循环**来控制二维数组中的元素的索引值。

```
var arr = [[12, 59, 66], [100, 888]];
var sum = 0;
for (var i = 0; i < arr.length; i++) {    // 遍历arr数组
    for (var j = 0; j < arr[i].length; j++) { // 遍历arr[i]数组
        sum += arr[i][j];                  // 二维数组元素累加
    }
}
console.log(sum); // 输出结果 : 1125
```

3.6 二维数组

3

二维数组转置

二维数组的转置指的是将二维数组横向元素保存为纵向元素。

转置前arr

```
[  
  ['a', 'b', 'c'],  
  ['d', 'e', 'f'],  
  ['g', 'h', 'i'],  
  ['j', 'k', 'l'],  
]
```

转置后res

```
[  
  ['a', 'd', 'g', 'j'],  
  ['b', 'e', 'h', 'k'],  
  ['c', 'f', 'i', 'l'],  
]
```

转置规律

```
res[0][0] = arr[0][0]
```

```
res[0][1] = arr[1][0]
```

```
res[0][2] = arr[2][0]
```

```
res[0][3] = arr[3][0]
```

3.6 二维数组

3

二维数组转置

案例需求：完成二维数组的转置

```
var arr = [
  ['a', 'b', 'c'], ['d', 'e', 'f'], ['g', 'h', 'i'], ['j', 'k', 'l']
];
var res = [];
for (var i = 0; i < arr[0].length; i++) {
  res[i] = [];
  for(var j = 0; j < arr.length; j++) {
    res[i][j] = arr[j][i];      // 为二维数组赋值
  }
}
console.log(res);
```



本章总结

本章首先介绍了JavaScript流程控制中的循环结构的相关内容，然后讲解了数组的创建、访问、遍历等基础操作，通过案例巩固加强读者对数组的认识。最后讲解二维数组的创建并通过案例的形式演示二维数组求和与转置，深化对数组的理解和运用。